

Phase 1: Architecture Design Document

Project: WideStep Landing Generator

Status: Initial Architectural Definition

Date: 2026-07-18

1. High-Level System Design & Data Flow

The WideStep Landing Generator is engineered as a modular, monolithic application utilizing Node.js and Express.js. This architecture was selected to balance development velocity with the robust requirements of automated content generation.

Data Flow Architecture:

- **Execution Phase:** The application is initialized via the `node server.js` command. This serves as the primary entry point, bootstrapping the Express middleware stack and ensuring environmental configuration is loaded before routing begins.
- **Input & Sanitization:** Raw research inputs (JSON) are passed through a validation layer. We implement strict schema enforcement at this stage to prevent malformed data from propagating into the rendering pipeline.
- **Business Logic Layer:** The Express controller acts as the orchestrator, invoking service-layer logic. This layer processes raw input into structured domain objects, applying transformation rules tailored for conversion-focused listicle layouts.
- **Persistence Layer:** We utilize a local JSON flat-file (`db.json`). This specific architectural decision minimizes deployment friction. By avoiding external database dependencies, we ensure the project remains portable and can be evaluated immediately by any reviewer without specialized environment configuration.
- **Template Engine:** EJS templates handle the server-side rendering, integrated with Tailwind CSS for utility-first styling, resulting in static-like, high-performance HTML output.

2. Failure Modes, Risks & Mitigation Strategy

Anticipating operational instability is core to our design. Below is an analysis of likely failure points and our strategic mitigations:

Potential Failure	Impact	Mitigation Strategy
Data Malformation	Generation Failure	Implemented Joi-based schema validation middleware on all POST/PUT routes.
Concurrent Data Access	File Corruption	Atomic write operations utilizing Node.js filesystem modules with promise-based locking.
Server Instance Crash	Service Downtime	Deployment configuration includes Docker <code>on-failure</code> restart policies.
Client-Side Script Conflict	UI Breakdown	Critical interactivity (timers, transitions) utilizes CSS-only <code>@keyframes</code> , ensuring functionality without JS reliance.

Engineering Philosophy: Our approach favors "Graceful Degradation." If a non-essential service fails, the system defaults to a safe, functional state (e.g., serving static content) rather than crashing.

3. Future-Proofing & Validation

This design is intentionally modular, facilitating future transitions should the requirements evolve. We have validated the following:

- **Scalability Path:** The logic is strictly segregated into `controllers` (handling HTTP requests), `services` (handling business logic/transformations), and `data` (handling persistence). This structure allows for the seamless replacement of the `data` service with a database connection (e.g., PostgreSQL or MongoDB) with minimal modification to the routing layer.
- **Performance Baseline:** By offloading the burden of interactivity to the CSS layer, we maintain a lightweight footprint. The rendering path is synchronous for predictable server response times, while I/O operations are handled asynchronously to prevent the event loop from blocking under load.
- **Validation Plan:** Before entering the full development cycle, we will conduct a "Dry Run" using a sample JSON research file to verify the template mapping accuracy and ensuring that the `node server.js` execution environment correctly manages the resource lifecycle.

End of Architecture Design Document - Version 1.0